

Will AI Be Admitted ?

Predicting 30 Day Hospital Readmission

Submitted in Partial Fulfillment Of the Requirement for the “ “

Prepared by:

Sumaya, Melfred

Date of Submission:

I. Introduction

1.1 Problem Framing

Unplanned hospital readmission within thirty days of discharge is widely used as an indicator of care quality and continuity, because a patient returning so soon after discharge may signal that recovery was incomplete, that follow-up was insufficient, or that the discharge itself was premature. The scale of the problem is substantial. National hospital data place the all-cause thirty-day readmission rate at approximately 14 readmissions per 100 index admissions, rising to 16.9 percent among older patients covered by Medicare and falling to 8.7 percent among privately insured stays, indicating that risk is concentrated in identifiable patient groups rather than spread evenly (Agency for Healthcare Research and Quality [AHRQ], 2021). Each of these readmissions occupies a bed that another patient could have used, and a share of them is avoidable. Compounding this, the difficulty facing hospitals is not a lack of awareness of the problem but a lack of capacity to act on it, since discharge teams handle a volume of patients that exceeds what they can monitor closely, and follow-up is therefore distributed evenly or informally rather than concentrated where it would prevent the most harm.

Research has established that a meaningful share of these readmissions is preventable, and has identified clinical and care-related predictors such as emergency readmission, multiple medications at discharge, and the presence of hospital-acquired infection, which indicates that readmission risk is patterned rather than random. This capstone therefore focuses on whether that risk can be predicted reliably enough to be useful at the point of care. Stated as a data science problem, the task is to estimate the probability that a patient will be readmitted within thirty days, using only information already available at the moment of discharge. The proposed solution is a predictive model, delivered through a web application, that returns a readmission risk estimate for an individual patient at the time of discharge. The output is not a diagnosis and does not replace clinical judgement. It is an open source tool that allows a care team to direct its limited follow-up effort toward the patients most likely to return, turning an informal decision into one supported by evidence from historical discharge records.

1.2 Scope and Target Variable

The project is scoped as a binary classification task. The target variable is readmitted, coded 0 where the patient was not readmitted within thirty days of discharge and 1 where a readmission occurred within that window. Classification is the appropriate formulation because the outcome of interest is a two-state event rather than a continuous quantity, which rules out regression, and because the outcome is already recorded for every historical discharge in the dataset, which rules out clustering. The model therefore learns from labelled examples rather than searching for structure in unlabelled data.

Although the underlying task is classification, the model output used in practice is the predicted probability of readmission rather than the hard 0 or 1 label. This allows patients to be ranked by risk so that follow-up effort can be allocated from the highest risk downward, and it

allows the decision threshold to be adjusted later according to how much follow-up capacity a care team actually has, without retraining the model.

1.3 Success Metrics and Performance Targets

Table 1.3.1

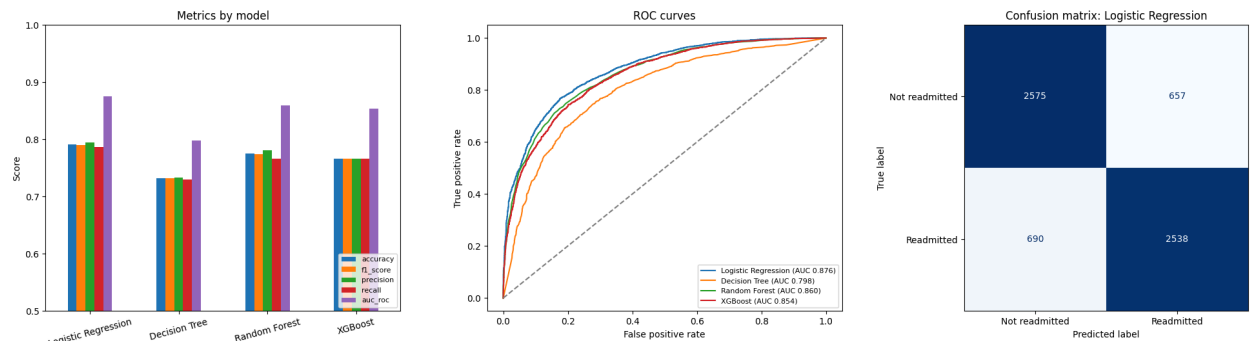
Model Performance Comparison: Metric Scores, ROC Curves, and Confusion Matrix of the Selected Model

Model	Accuracy	F1 Score	Precision	Recall	AUC-ROC
Logistic Regression	0.7915	0.7903	0.7944	0.7862	0.8755
Random Forest	0.7759	0.7737	0.7808	0.7667	0.8600
XGBoost	0.7664	0.7662	0.7663	0.7661	0.8538
Decision Tree	0.7327	0.7319	0.7336	0.7302	0.7979

The table shows that Logistic Regression achieved the highest AUC-ROC at 0.8755 and the highest recall at 0.7862. Random Forest and XGBoost performed competitively but did not surpass it on either metric, while the single Decision Tree performed worst across all five measures, which is consistent with its tendency to overfit without ensembling. Logistic Regression was therefore selected as the final model, with the added advantage that its coefficients are directly interpretable, which matters in a clinical setting where a care team must be able to explain why a patient was flagged.

Figure 1.3.2

Comparative Evaluation Results of the Four Candidate Models



The figure presents three views of the same evaluation. The bar chart compares all four models across accuracy, F1, precision, recall, and AUC-ROC, showing Logistic Regression leading on every metric and Decision Tree trailing on every metric. The ROC curves express the same result as separability, with Logistic Regression at AUC 0.876 lying closest to the upper-left corner, followed by Random Forest at 0.860 and XGBoost at 0.854, while Decision Tree at 0.798 sits nearest the diagonal reference line that represents random guessing. The confusion matrix for the selected model shows 2,538 readmissions correctly identified and 690 missed, against 2,575 correct non-readmissions and 657 patients flagged who were not in fact readmitted. The errors are therefore close to balanced, with slightly more missed readmissions than false alarms.

II. Data Collect and Understanding

2.1 Datasets

Figure 2.1.1

Data Source and Result

```
# Mount Dataset from Google Drive

file_path = '/content/drive/MyDrive/CapstoneDataset/hospital_readmission.csv'

print("Loading dataset from Google Drive...")
df = pd.read_csv(file_path)
print("Dataset loaded successfully!")
print(f"Dataset shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
print(f"First 5 rows:")
print(df.head())
```

```
... Loading dataset from Google Drive...
Dataset loaded successfully!
Dataset shape: (32300, 10)
Columns: ['age', 'length_of_stay_days', 'num_diagnoses', 'num_medications', 'prev_admissions', 'glucose_level', 'bmi', 'has_diabetes', 'discharge_type', 'readmitted_30days']

First 5 rows:
   age  length_of_stay_days  num_diagnoses  num_medications  prev_admissions \
0  22.0                28.0              2.0              13.0              4.0
1  44.0                12.0              7.0              11.0              2.0
2  34.0                15.0              9.0              16.0              6.0
3  39.0                15.0              5.0              16.0              6.0
4  62.0                24.0              1.0               6.0              6.0

   glucose_level  bmi  has_diabetes  discharge_type  readmitted_30days
0      89.1  48.4              1.0              2.0              0
1     254.8  23.9              1.0              0.0              1
2     278.2  16.7              0.0              1.0              1
3     208.6  18.2              0.0              1.0              1
4     233.1  44.0              0.0              0.0              1
```

This study uses the Hospital Readmission Prediction Dataset by Abhay Panchal, sourced from Kaggle, comprising 32,300 patient records. It contains clinical and demographic variables covering age, length of stay, prior admissions, diagnosis and medication counts, BMI, glucose level, diabetes status, and discharge type, with 30-day readmission as the binary outcome.

As shown in Figure 2.1.1, the dataset was mounted from Google Drive and loaded into google colab environment, where its structure was verified before processing. The output confirms 32,300 records across 10 columns, comprising nine predictor variables and the target variable readmitted_30days, with each row representing a single hospital discharge.

2.2 Feature Types, Missing Values, and Outliers

Figure 2.2.1

Dataset Overview and Result

```
# Dataset Overview (Features type, missing value)

print("Dataset Overview")
print(f"Total records: {len(df)}")
print(f"Features: {df.columns.tolist()}")
print(f"\nMissing values:\n{df.isnull().sum()}")
print(f"\nData types:\n{df.dtypes}")
print(f"\nDescriptive statistics:\n{df.describe()}")

# Check target variable distribution
target_col = 'readmitted_30days'
if target_col in df.columns:
    print(f"\nTarget column '{target_col}' found!")
    print(f"Readmission rate: {df[target_col].mean():.2%}")
    print(f"Class distribution:\n{df[target_col].value_counts()}")
else:
    print(f"\nWarning: Target column '{target_col}' not found. Available columns: {df.columns.tolist()}")
```

Dataset Overview
Total records: 32300
Features: ['age', 'length_of_stay_days', 'num_diagnoses', 'num_medications', 'prev_admissions', 'glucose_level', 'bmi', 'has_diabetes', 'discharge_type', 'readmitted_30days']

Missing values:

age	647
length_of_stay_days	648
num_diagnoses	645
num_medications	646
prev_admissions	646
glucose_level	646
bmi	646
has_diabetes	647
discharge_type	645
readmitted_30days	0
dtype:	int64

Data types:

age	object
length_of_stay_days	float64
num_diagnoses	float64
num_medications	float64
prev_admissions	float64
glucose_level	float64
bmi	float64
has_diabetes	float64
discharge_type	float64
readmitted_30days	int64
dtype:	object

Descriptive statistics:

	length_of_stay_days	num_diagnoses	num_medications	prev_admissions
count	31652.000000	31655.000000	31654.000000	31654.000000
mean	15.000846	7.529521	10.05418	4.500916
std	8.394087	4.023350	5.57273	2.865609
min	1.000000	1.000000	1.00000	0.000000
25%	8.000000	4.000000	5.00000	2.000000
50%	15.000000	8.000000	10.00000	5.000000
75%	22.000000	11.00000	15.00000	7.000000
max	29.000000	14.000000	200.00000	9.000000

	glucose_level	bmi	has_diabetes	discharge_type
count	31654.000000	31654.000000	31653.000000	31655.000000
mean	179.799055	32.571947	0.355195	0.998136
std	69.854520	10.098815	0.478580	0.815959
min	60.000000	-5.000000	0.000000	0.000000
25%	119.000000	23.900000	0.000000	0.000000
50%	179.900000	32.600000	0.000000	1.000000
75%	240.700000	41.200000	1.000000	2.000000
max	900.000000	50.000000	1.000000	2.000000

readmitted_30days

count	32300.000000
mean	0.499659
std	0.500008
min	0.000000
25%	0.000000
50%	0.000000
75%	1.000000
max	1.000000

Target column 'readmitted_30days' found!
Readmission rate: 49.97%
Class distribution:
readmitted_30days
0 16161
1 16139
Name: count, dtype: int64

Figure 2.1.2 exhibits code execution that shows the dataset overview routine, which reports the total number of records, the full list of features, the count of missing values per column, the data type of each column, and the descriptive statistics for all numeric variables. The same cell then verifies the presence of the target column `readmitted_30days` and reports its distribution, expressing the readmission

rate as a percentage and the class counts in absolute terms, so that the balance between readmitted and non-readmitted cases is established before any modelling decision is made.

Results exhibited the target are almost perfectly balanced, with 16,161 non-readmitted and 16,139 readmitted cases, giving a readmission rate of 49.97 percent. This balance means resampling is unnecessary and accuracy remains a meaningful metric.

2.3 Data Dictionary

Figure 2.3.1

Data Dictionary Structure and Result

```

# Data dictionary
target_col = 'readmitted_30days'

DICTIONARY = [
    # variable, type, unit, allowed values, description
    ('age', 'Numeric (continuous)', 'years', '0 to 110', 'Patient age at admission'),
    ('length_of_stay_days', 'Numeric (count)', 'days', '1 to 365', 'Days from admission to discharge'),
    ('num_diagnoses', 'Numeric (count)', 'count', '1 to 50', 'Number of diagnoses recorded during the stay'),
    ('num_medications', 'Numeric (count)', 'count', '0 to 50', 'Number of medications prescribed'),
    ('prev_admissions', 'Numeric (count)', 'count', '0 to 50', 'Number of previous hospital admissions'),
    ('glucose_level', 'Numeric (continuous)', 'mg/dL', '40 to 600', 'Blood glucose level'),
    ('bmi', 'Numeric (continuous)', 'kg/m2', '10 to 80', 'Body mass index'),
    ('has_diabetes', 'Binary', 'none', '0 = no, 1 = yes', 'Diabetes diagnosis'),
    ('discharge_type', 'Categorical (coded)', 'none', '0, 1, 2', 'Discharge category code'),
    (target_col, 'Binary (target)', 'none', '0 = no, 1 = yes', 'Readmitted within 30 days of discharge'),
]

data_dictionary = pd.DataFrame(
    DICTIONARY, columns=['variable', 'type', 'unit', 'allowed_values', 'description'])

observed, missing_count = [], []
for v in data_dictionary['variable']:
    if v in df.columns:
        values = pd.to_numeric(df[v].astype(str).str.replace('_', ''), errors='coerce')
        observed.append(f"{values.min():g} to {values.max():g}")
        missing_count.append(int(df[v].isna().sum()))
    else:
        observed.append('not in dataset')
        missing_count.append(0)

data_dictionary['observed_range'] = observed
data_dictionary['missing'] = missing_count
data_dictionary = data_dictionary[
    ['variable', 'type', 'unit', 'allowed_values', 'observed_range', 'missing', 'description']]

data_dictionary.to_csv('data_dictionary.csv', index=False)
display(data_dictionary)

```

	variable	type	unit	allowed_values	observed_range	missing	description
0	age	Numeric (continuous)	years	0 to 110	18 to 150	647	Patient age at admission
1	length_of_stay_days	Numeric (count)	days	1 to 365	1 to 29	648	Days from admission to discharge
2	num_diagnoses	Numeric (count)	count	1 to 50	1 to 14	645	Number of diagnoses recorded during the stay
3	num_medications	Numeric (count)	count	0 to 50	1 to 200	646	Number of medications prescribed
4	prev_admissions	Numeric (count)	count	0 to 50	0 to 9	646	Number of previous hospital admissions
5	glucose_level	Numeric (continuous)	mg/dL	40 to 600	60 to 900	646	Blood glucose level
6	bmi	Numeric (continuous)	kg/m2	10 to 80	-5 to 50	646	Body mass index
7	has_diabetes	Binary	none	0 = no, 1 = yes	0 to 1	647	Diabetes diagnosis
8	discharge_type	Categorical (coded)	none	0, 1, 2	0 to 2	645	Discharge category code
9	readmitted_30days	Binary (target)	none	0 = no, 1 = yes	0 to 1	0	Readmitted within 30 days of discharge

The data dictionary defines each of the ten variables by type, unit, allowed values, and meaning, satisfying the requirement for a documented variable reference. The code pairs each declared specification with the values actually observed in the dataset, computing the observed range and missing

count per variable, so that the dictionary functions as a validation check rather than a description alone. The result is exported as a csv file for reuse.

III. Data Preprocessing, Applied EDA, & Feature Engineering

3.1 Data Cleaning

Figure 3.1.1

Handling of Missing Values and Outliers

```
# Data Cleaning (Nulls and Outliers)

print("Check for missing values: ")
for col in df.columns:
    if df[col].isnull().sum() > 0:
        if df[col].dtype in ['int64', 'float64']:
            df[col] = df[col].fillna(df[col].median())
            print(f" {col}: Filled missing values with median")
        else:
            df[col] = df[col].fillna(df[col].mode()[0])
            print(f" {col}: Filled missing values with mode")

if target_col in df.columns:
    df = df.dropna(subset=[target_col])
    print(f"Removed rows with missing target. New shape: {df.shape}")

def cap_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[column] = df[column].clip(lower, upper)
    return df

numeric_cols = df.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    if col != target_col:
        df = cap_outliers(df, col)
print(f"\nOutliers capped for {len(numeric_cols)-1} numeric columns")

... Check for missing values:
age: Filled missing values with mode
length_of_stay_days: Filled missing values with median
num_diagnoses: Filled missing values with median
num_medications: Filled missing values with median
prev_admissions: Filled missing values with median
glucose_level: Filled missing values with median
bmi: Filled missing values with median
has_diabetes: Filled missing values with median
discharge_type: Filled missing values with median
Removed rows with missing target. New shape: (32300, 10)

Outliers capped for 8 numeric columns
```

Cleaning of data starts with the missing values. Numeric columns were filled with the median and categorical columns with the mode, thus the shape of each distribution stays the same and no records are lost. Rows with a missing target were then dropped, but none were removed because the target column had no missing values across all 32,300 records. Cleaning ends with the outliers. In the eight numeric predictors, values falling beyond 1.5 times the interquartile range were capped at the nearest limit instead of deleted. This corrects impossible entries such as the BMI of -5 and the medication count of 200 while keeping the patient records intact.

3.2 Engineer features

Figure 3.2.1

Feature Engineering Implementation

```
# Check contributing features (Binning and Domain Features)
target_col = 'readmitted_30days'

if 'age' in df.columns:
    df['age'] = df['age'].astype(str).str.replace('_', '').str.replace(' ', '')
    df['age'] = pd.to_numeric(df['age'], errors='coerce')
    df['age'] = df['age'].fillna(df['age'].median())

if 'length_of_stay_days' in df.columns:
    df['prolonged_stay'] = (df['length_of_stay_days'] >= 7).astype(int)

if 'prev_admissions' in df.columns:
    df['frequent_admissions'] = (df['prev_admissions'] >= 3).astype(int)

if 'num_diagnoses' in df.columns:
    df['high_diagnoses'] = (df['num_diagnoses'] >= 10).astype(int)

if 'num_medications' in df.columns:
    df['high_medications'] = (df['num_medications'] >= 15).astype(int)

if 'glucose_level' in df.columns:
    df['high_glucose'] = (df['glucose_level'] >= 200).astype(int)

if 'bmi' in df.columns:
    df['obese'] = (df['bmi'] >= 30).astype(int)

df['risk_score'] = (
    0.25 * (df['prev_admissions'] / 10) +
    0.20 * (df['length_of_stay_days'] / 30) +
    0.15 * (df['num_diagnoses'] / 15) +
    0.15 * (df['num_medications'] / 20) +
    0.15 * (df['glucose_level'] / 300) +
    0.10 * (df['bmi'] / 50)
)

feature_cols = [
    'age',
    'length_of_stay_days',
    'num_diagnoses',
    'num_medications',
    'prev_admissions',
    'glucose_level', #Uses data from diabetes.
    'bmi',
    'has_diabetes',
    'discharge_type',
    'prolonged_stay',
    'frequent_admissions',
    'high_diagnoses',
    'high_medications',
    'high_glucose',
    'obese'
]

existing_features = [col for col in feature_cols if col in df.columns]
missing_features = [col for col in feature_cols if col not in df.columns]

if missing_features:
    print(f"Warning: These features are missing: {missing_features}")
    for col in missing_features:
        df[col] = 0

print(f"\nFeature engineering complete. Selected {len(feature_cols)} features.")
print(f"Features: {feature_cols}")

X = df[feature_cols]
y = df[target_col]
```

The code implementation is used for preparing the model inputs. It converts age from text to numbers, creates six yes or no flags from clinical cut-offs such as a 7-day stay and a BMI of 30, and builds a weighted risk_score. The 13 resulting columns form X, with readmitted_30days as the target y.

3.3 Applied EDA

Figure 3.3.1

Distribution and Correlation Analysis Code Implementation

```
1 # Feature Distribution, checking how features contribute to the output (Relationships)

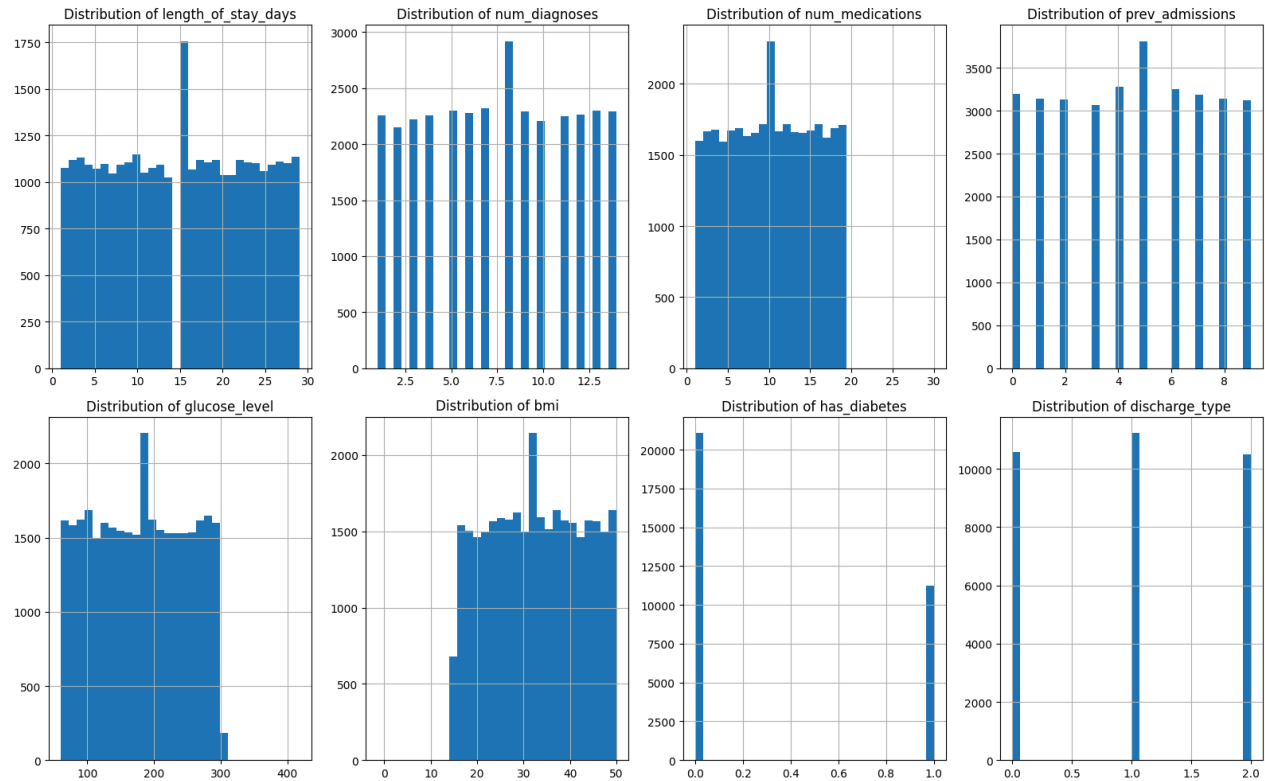
fig, axes = plt.subplots(2, 4, figsize=(16, 10))
numeric_features = [col for col in df.select_dtypes(include=[np.number]).columns if col != target_col][:8]
for i, feature in enumerate(numeric_features):
    ax = axes[i//4, i%4]
    df[feature].hist(bins=30, ax=ax)
    ax.set_title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()

# Correlation analysis
plt.figure(figsize=(14, 12))
corr_matrix = df.select_dtypes(include=[np.number]).corr()
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0, fmt='.2f', annot_kws={'size': 8})
plt.title('Feature Correlation Matrix')
plt.tight_layout()
plt.show()
```

The code plots histograms for the first eight numeric predictors to show the shape and spread of each distribution, then computes a correlation matrix across all numeric variables and renders it as a bar graph visuals. Together these establish how the features are distributed individually and how strongly they relate to one another and to the target before modelling begins.

Figure 3.3.2

Distribution of Numeric Features After Cleaning



In this graph, the distributions are close to uniform for most variables. Length of stay spreads evenly across 1 to 29 days, diagnosis counts across 1 to 14, and prior admissions across 0 to 9, with no variable showing a normal or skewed shape. About 36 percent of patients carry a diabetes diagnosis, and discharge type is concentrated in three coded values.

Two patterns come from the cleaning stage. The single tall spike in several plots is the result of median imputation, which piles the missing 2 percent onto one value, while the sharp drop at the upper end of glucose level and BMI marks the IQR cap.

3.4 Feature importance & explainability

Figure 3.4.1

Model-Based Explainability and Feature Contribution by BMI Group

```
# Model visualization using how BMI contributes to the output.(Feature Importance)

try:
    if hasattr(best_model, 'feature_importances_'):
        explainer = shap.TreeExplainer(best_model)
        shap_values = explainer.shap_values(X_test_scaled[:100])

        plt.figure(figsize=(12, 8))
        shap.summary_plot(shap_values, X_test_scaled[:100], feature_names=feature_cols, show=False)
        plt.title('SHAP Feature Importance Summary')
        plt.tight_layout()
        plt.show()
except Exception as e:
    print(f"SHAP visualization skipped: {e}")

if 'bmi' in df.columns:
    df_test = X_test.copy()
    df_test['predicted'] = best_model.predict(X_test_scaled)
    df_test['actual'] = y_test.values

    bmi_groups = pd.cut(df_test['bmi'], bins=[0, 18.5, 25, 30, 40, 100],
                        labels=['Underweight', 'Normal', 'Overweight', 'Obese', 'Severe Obese'])
    bmi_group_metrics = df_test.groupby(bmi_groups)['predicted'].mean()
    print("\nFAIRNESS ANALYSIS: Prediction rates by BMI group")
    print(bmi_group_metrics)

    overall_rate = df_test['predicted'].mean()
    print(f"\nOverall prediction rate: {overall_rate:.2%}")
    for group in bmi_group_metrics.index:
        group_rate = bmi_group_metrics[group]
        disparate_impact = group_rate / overall_rate if overall_rate > 0 else 1
        print(f"{group}: {group_rate:.2%} (DI: {disparate_impact:.2f})")

'''
FAIRNESS ANALYSIS: Prediction rates by BMI group
bmi
Underweight    0.392273
Normal         0.438961
Overweight     0.445583
Obese          0.493085
Severe Obese   0.595462
Name: predicted, dtype: float64

Overall prediction rate: 49.46%
Underweight: 39.23% (DI: 0.79)
Normal: 43.90% (DI: 0.89)
Overweight: 44.55% (DI: 0.90)
Obese: 49.31% (DI: 1.00)
Severe Obese: 59.55% (DI: 1.20)
```

This code implementation exhibits the fairness analysis for the selected model, structured in three stages. It first assembles a working copy of the test set and attaches the model's predictions alongside the actual outcomes, so that predicted and observed values can be compared within the same frame. It then bins BMI into five clinical bands, from underweight through severely obese, and computes the mean prediction rate within each band, giving the proportion of patients flagged as high risk per group. Finally, it divides each group's rate by the overall prediction rate to produce a disparate impact ratio, the standard measure of whether one group is flagged disproportionately relative to the population.

The results show the flagging rate rising steadily with BMI, from 39.23 percent among underweight patients to 59.55 percent among the severely obese, against an overall rate of 49.46 percent. Disparate impact ranges from 0.79 to 1.20, placing the underweight group marginally below the 0.80 threshold conventionally used to signal disparity.

3.5 Feature selection

Figure 3.5.1

Train-Test Split Code Implementation

```
# Train-test split. It uses features with highest significance according to KBest (Feature selection and scaling)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

selector = SelectKBest(score_func=f_classif, k=min(10, len(feature_cols)))
X_train_selected = selector.fit_transform(X_train_scaled, y_train)
X_test_selected = selector.transform(X_test_scaled)
selected_features = np.array(feature_cols)[selector.get_support()]

print(f"\nSelected features ({len(selected_features)}): {selected_features}")

'''
Selected features (10): ['age' 'length_of_stay_days' 'num_diagnoses' 'num_medications'
 'prev_admissions' 'discharge_type' 'prolonged_stay' 'frequent_admissions'
 'high_diagnoses' 'high_medications']
'''
```

The data is split 80/20 with stratification on the target, preserving the class balance in both sets, and the random state is fixed at 42 for reproducibility. Scaling is fitted on the training set only and then applied to the test set, so no information from the test data influences the transformation.

3.6 Dimensionality reduction

Figure 3.6.1

Dimensionality Reduction Using PCA Code Implementation

```
# Dimensionality Reduction using PCA

from sklearn.base import clone
from sklearn.decomposition import PCA

pca_full = PCA(random_state=SEED)
pca_full.fit(X_train_scaled)

explained = pca_full.explained_variance_ratio_
cumulative = np.cumsum(explained)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].bar(range(1, len(explained) + 1), explained * 100,
            color='#7043a1', alpha=0.8)
axes[0].set_xlabel('Principal Component')
axes[0].set_ylabel('Explained Variance (%)')
axes[0].set_title('Scree Plot - Individual Component Variance')
axes[0].set_xticks(range(1, len(explained) + 1))

axes[1].plot(range(1, len(cumulative) + 1), cumulative * 100,
            marker='o', color='#35104f')
axes[1].axhline(95, ls='--', color='grey', label='95% threshold')
axes[1].set_xlabel('Number of Components')
axes[1].set_ylabel('Cumulative Explained Variance (%)')
axes[1].set_title('Cumulative Variance Retained')
axes[1].legend()

plt.tight_layout()
plt.show()

n_components_95 = int(np.argmax(cumulative >= 0.95) + 1)
print(f"Components needed for 95% variance: {n_components_95} of {len(feature_cols)}")
print(f"Variance retained by {n_components_95} components: {cumulative[n_components_95-1]:.2%}")

pca = PCA(n_components=n_components_95, random_state=SEED)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

pca_2d = PCA(n_components=2, random_state=SEED)
X_2d = pca_2d.fit_transform(X_train_scaled)
```

```

pca = PCA(n_components=n_components_95, random_state=SEED)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

pca_2d = PCA(n_components=2, random_state=SEED)
X_2d = pca_2d.fit_transform(X_train_scaled)

plt.figure(figsize=(8, 6))
for label, colour, name in [(0, '#388e3c', 'Not readmitted'),
                             (1, '#c62828', 'Readmitted')]:
    mask = (y_train.values == label)
    plt.scatter(X_2d[mask, 0], X_2d[mask, 1],
                s=12, alpha=0.4, c=colour, label=name)
plt.xlabel(f'PC1 ({pca_2d.explained_variance_ratio_[0]:.1%})')
plt.ylabel(f'PC2 ({pca_2d.explained_variance_ratio_[1]:.1%})')
plt.title('PCA 2D Projection - Readmission Classes')
plt.legend()
plt.tight_layout()
plt.show()

pca_model = clone(best_model)
pca_model.fit(X_train_pca, y_train)

y_prob_pca = pca_model.predict_proba(X_test_pca)[:, 1]
auc_pca = roc_auc_score(y_test, y_prob_pca)
acc_pca = accuracy_score(y_test, pca_model.predict(X_test_pca))

comparison_pca = pd.DataFrame({
    'representation': (f'Full features ({len(feature_cols)})',
                       f'PCA ({n_components_95} components)'),
    'model': [best_model_name, best_model_name],
    'accuracy': [results[best_model_name]['accuracy'], acc_pca],
    'auc_roc': [results[best_model_name]['auc_roc'], auc_pca],
})
display(comparison_pca.round(4))

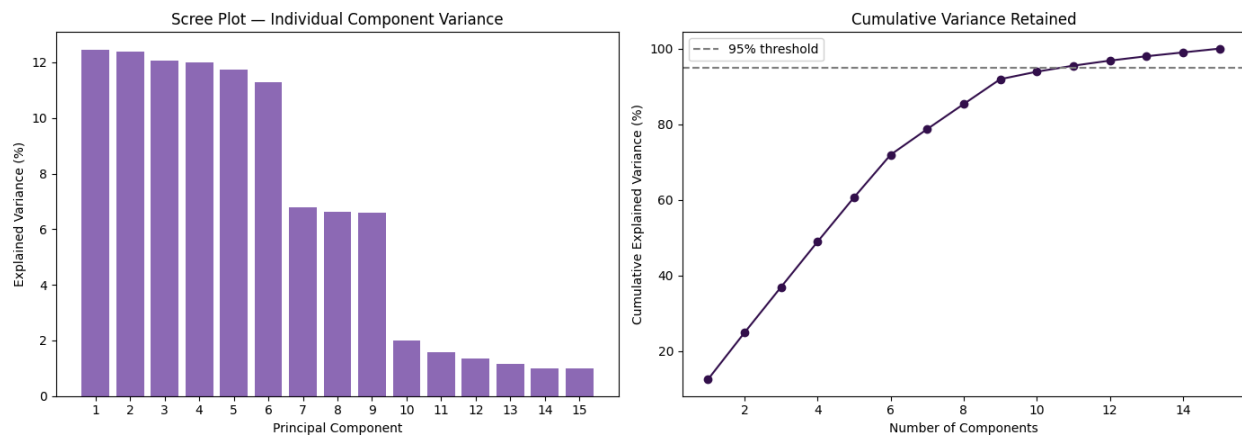
print(f"\nInterpretation: PCA retains {cumulative[n_components_95-1]:.1%} of variance "
      f"with {n_components_95} components. AUC change: "
      f"{auc_pca - results[best_model_name]['auc_roc']:+.4f}")

```

The code implementation fits PCA on the scaled training data, computes the individual and cumulative explained variance, and determines how many components are needed to retain 95 percent of the variance. It then projects the data onto that number of components, produces a separate two-component projection for visualisation, and retrains the selected model on the reduced representation so that the full and reduced feature sets can be compared on accuracy and AUC.

Figure 3.6.2

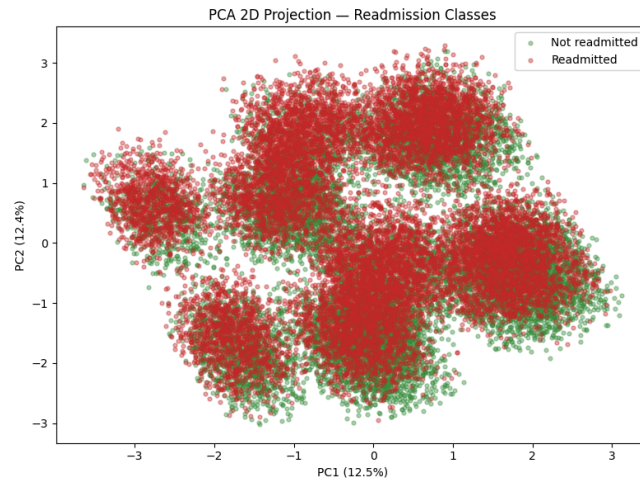
Scree Plot and Cumulative Explained Variance



Variance is spread evenly across the components rather than concentrated in a few. The first six components each explain roughly 11 to 12.5 percent, after which the contribution drops, and around 11 components are required to reach the 95 percent threshold out of 15. This indicates little redundancy among the features, so dimensionality reduction offers minimal benefit here.

Figure 3.6.3

Two-Component PCA Projection by Readmission Class



The projection shows the two classes heavily overlapping, with readmitted and non-readmitted patients occupying the same regions and no visible separation along either component. This is expected, since the first two components together account for under 25 percent of the total variance, and it confirms that readmission risk is not linearly separable in two dimensions and requires the full model to detect.

IV. Model Implementation

4.1 Model Experimentation & 4.2 Model Evaluation

Figure 4.1.1

Model Training and Evaluation Code Implementation

```
# Model Training, Evaluation metrics visualization.
# Uses 4 out 5 models mentioned in the assessment
# random_state = 42 is used because the use of random seed helps in reproducibility. This ensures that no matter who runs the program, it will provide consistent outputs.

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced'),
    'Decision Tree': DecisionTreeClassifier(random_state=42, max_depth=10, class_weight='balanced'),
    'Random Forest': RandomForestClassifier(random_state=42, n_estimators=100, class_weight='balanced'),
    'XGBoost': XGBClassifier(random_state=42, eval_metric='logloss', use_label_encoder=False, scale_pos_weight=len(y_train[y_train==0])/len(y_train[y_train==1]))
}

results = {}
best_model = None
best_auc = 0
best_model_name = None

print("\nMODEL TRAINING AND EVALUATION")
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1] if hasattr(model, 'predict_proba') else None

    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob) if y_prob is not None else None

    results[name] = {
        'accuracy': accuracy,
        'f1_score': f1,
        'precision': precision,
        'recall': recall,
        'auc_roc': auc
    }

    print(f"\n{name}:")
    print(f" Accuracy: {accuracy:.4f}")
    print(f" F1 Score: {f1:.4f}")
    print(f" Precision: {precision:.4f}")
    print(f" Recall: {recall:.4f}")
    print(f" AUC-ROC: {auc:.4f} if auc else " AUC-ROC: N/A")

    if auc and auc > best_auc:
        best_auc = auc
        best_model = model
        best_model_name = name

# Visualization of the comparison between models.
from sklearn.metrics import roc_curve, ConfusionMatrixDisplay

results_df = pd.DataFrame(results).T

fig, axes = plt.subplots(1, 3, figsize=(20, 5.5))

# 1) All metrics side by side
results_df.plot.bar(ax=axes[0], rot=15)
axes[0].set_ylim(0.5, 1.0)
axes[0].set_title('Metrics by model')
axes[0].set_ylabel('Score')
axes[0].legend(fontsize=8, loc='lower right')

# 2) ROC curves for every model
for name, model in models.items():
    y_prob = model.predict_proba(X_test_scaled)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    axes[1].plot(fpr, tpr, label=f'{name} (AUC {results[name]["auc_roc"]:.3f})')
axes[1].plot([0, 1], [0, 1], ls='--', color='grey')
axes[1].set_xlabel('False positive rate')
axes[1].set_ylabel('True positive rate')
axes[1].set_title('ROC curves')
axes[1].legend(fontsize=8)

# 3) Confusion matrix for the winning model
ConfusionMatrixDisplay.from_predictions(
    y_test, best_model.predict(X_test_scaled),
    display_labels=['Not readmitted', 'Readmitted'],
    cmap='Blues', ax=axes[2], colorbar=False)
axes[2].set_title(f'Confusion matrix: {best_model_name}')

plt.tight_layout()
plt.show()
```

The code trains four supervised classifiers on the same scaled training data and the same split, so the comparison between them is fair: Logistic Regression, Decision Tree, Random Forest, and XGBoost, each with `random_state = 42` for reproducibility and class weighting applied. Every model is evaluated on the held-out test set using accuracy, F1, precision, recall, and AUC-ROC, with the results stored in a single dictionary and the highest AUC retained as the selected model. The cell then produces the three

comparison visualisations, namely the metric bar chart, the ROC curves, and the confusion matrix for the winning model, which are presented in Section 1.3 as Figure 1.3.1 together with the results table.

4.3 Reproducibility and Model Artefacts

Figure 4.3.1

Saving of Trained Model and Configuration Artefacts

```
# Model saving.
# For measuring a scale of 0 (no readmission) to 1 (will be readmitted), AUC performs well as it follows a ranking order (low, moderate, and high).

with open('readmission_model.pkl', 'wb') as f:
    pickle.dump(best_model, f)

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

feature_info = {
    'features': feature_cols,
    'selected_features': selected_features.tolist(),
    'model_type': best_model_name,
    'metrics': results[best_model_name] if best_model_name else {},
    'dataset_shape': df.shape,
    'readmission_rate': float(y.mean()),
    'feature_columns': feature_cols,
    'n_features': len(feature_cols)
}

with open('feature_info.json', 'w') as f:
    json.dump(feature_info, f)

print(f"\nBest model: {best_model_name} (AUC: {best_auc:.4f})")
print("Model artifacts saved successfully!")
print(f"Number of features: {len(feature_cols)}")

...
Best model: Logistic Regression (AUC: 0.8755)
Model artifacts saved successfully!
Number of features: 15
```

The selected model and its fitted scaler are serialised to `readmission_model.pkl` and `scaler.pkl`, so predictions can be reproduced without retraining. A companion `feature_info.json` records the full feature list, the features identified by `SelectKBest`, the model type, its evaluation metrics, the dataset shape, and the observed readmission rate, giving a complete configuration record alongside the binaries.

The output confirms Logistic Regression as the selected model with an AUC of 0.8755 across 15 features. Together with the fixed random state set at the start of the notebook, these artefacts satisfy the reproducibility requirement.

V. Critical Thinking: Ethical AI & Bias Auditing

5.1 Explain model decisions (SHAP, LIME, PDP, ICE)

SHAP code implementation

```
# SHAP explanations (matches the actual training setup)
import numpy as np
import pandas as pd
import shap
import matplotlib.pyplot as plt

RANDOM_STATE = 42

sel_names = list(feature_cols)

Xtr_sel = pd.DataFrame(X_train_scaled, columns=sel_names)
Xte_sel = pd.DataFrame(X_test_scaled, columns=sel_names)

sample_pos = np.random.RandomState(RANDOM_STATE).choice(
    len(Xte_sel), size=min(500, len(Xte_sel)), replace=False
)
X_explain = Xte_sel.iloc[sample_pos]

try:
    if isinstance(best_model, LogisticRegression):
        explainer = shap.LinearExplainer(
            best_model,
            shap.sample(Xtr_sel, 100, random_state=RANDOM_STATE)
        )
    else:
        explainer = shap.TreeExplainer(best_model)
        sv = explainer(X_explain)
except Exception as e:
    print(f'Fast explainer failed ({e}); using model-agnostic explainer (slower).')
    predict_pos = lambda data: best_model.predict_proba(data)[:, 1]
    explainer = shap.Explainer(
        predict_pos, shap.sample(Xtr_sel, 100, random_state=RANDOM_STATE)
    )
    sv = explainer(X_explain.iloc[:200])
    sample_pos = sample_pos[:200]
    X_explain = X_explain.iloc[:200]

if sv.values.ndim == 3:
    sv = sv[:, :, 1]

# Beeswarm
shap.plots.beeswarm(sv, max_display=len(sel_names), show=False)
plt.title(f'SHAP summary: {best_model_name}')
plt.tight_layout()
plt.show()

# Mean | SHAP |
shap_importance = (
    pd.Series(np.abs(sv.values).mean(axis=0), index=sel_names)
    .sort_values(ascending=False)
)
display(shap_importance.round(4).to_frame('mean_abs_shap'))

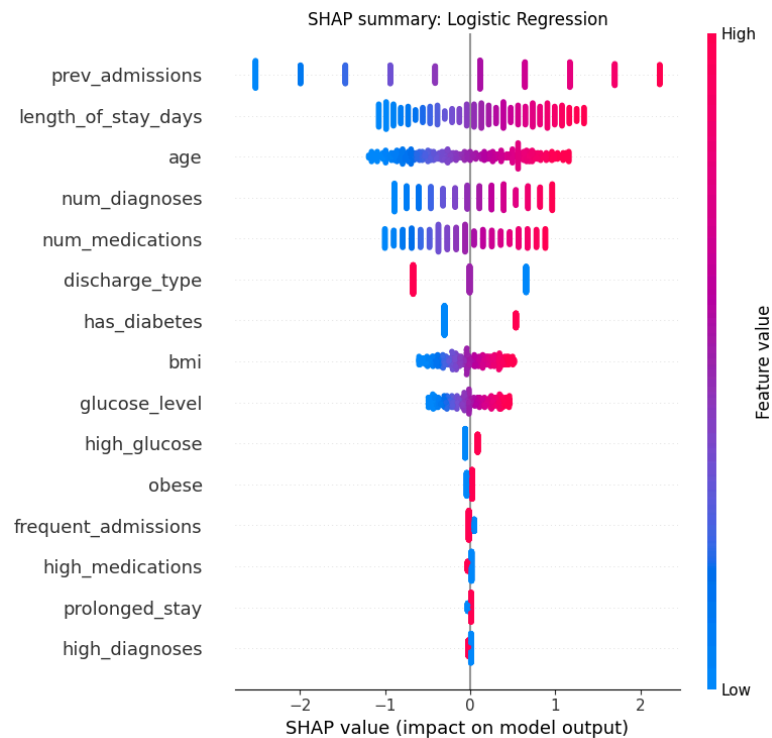
# Local explanations
probs_sample = best_model.predict_proba(X_explain)[:, 1]
for label, pos in [
    ('Highest predicted risk', int(np.argmax(probs_sample))),
    ('Lowest predicted risk', int(np.argmin(probs_sample))),
]:
    print(f'{label}: probability {probs_sample[pos]:.2f}')
    shap.plots.waterfall(sv[pos], max_display=10, show=False)
    plt.title(label)
    plt.tight_layout()
    plt.show()

# Odds ratios (Logistic Regression only)
if hasattr(best_model, 'coef_'):
    odds = pd.DataFrame(
        {
            'coefficient': best_model.coef_[0],
            'odds_ratio_per_1SD': np.exp(best_model.coef_[0]),
        },
        index=sel_names,
    ).sort_values('odds_ratio_per_1SD', ascending=False)
    display(odds.round(3))
```

The use of this code is about feature importance and explainability. It selects the explainer matching the chosen model, `LinearExplainer` for Logistic Regression and `TreeExplainer` otherwise, so the explanation runs against the model actually trained. SHAP values are computed on 500 test cases under a fixed random state, producing a beeswarm plot, a ranked table of mean absolute SHAP values, waterfall plots for the highest and lowest risk patients, and odds ratios per one standard deviation.

Figure 5.1.1

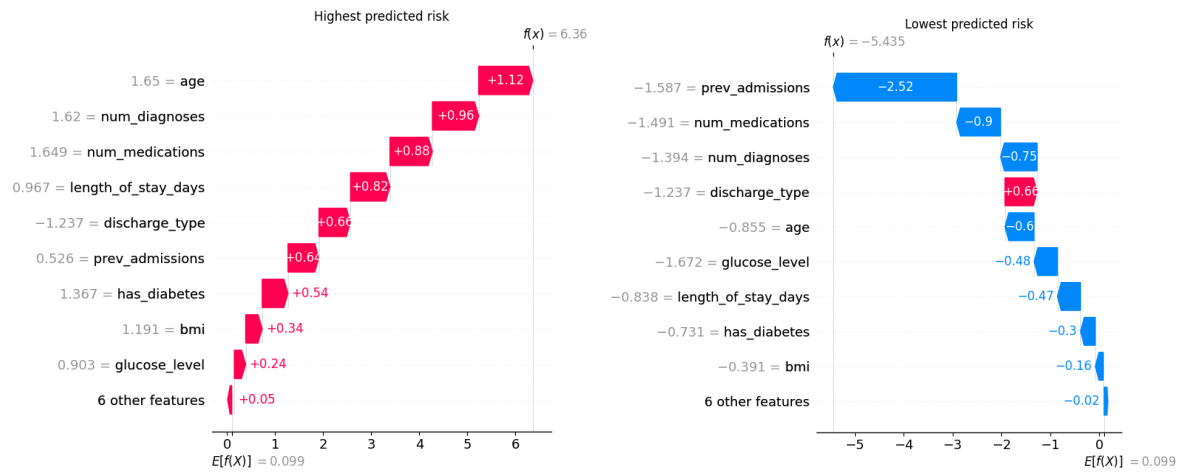
SHAP Summary Plot: Global Feature Influence



The plot ranks features by their average impact on the predicted probability. Prior admissions is the strongest driver by a wide margin, followed by length of stay, age, diagnosis count, and medication count. In each case, high values shown in red push predictions toward readmission, confirming that clinically heavier cases are flagged as higher risk. The binned flags such as high_glucose, obese, and prolonged_stay contribute almost nothing, since the continuous variables they were derived from are already in the model.

Figure 5.1.2 and 5.1.3

SHAP Waterfall Plots: Highest and Lowest Predicted Risk



The two plots trace how individual predictions are built from the same baseline of 0.099. For the highest risk patient, age, diagnosis count, medication count, and length of stay each push the score upward to a final value of 6.36. For the lowest risk patient, low prior admissions, medication count, and diagnosis count pull it down to -5.435. This demonstrates that the model can justify any single prediction, which matters in a clinical setting where a care team must be able to explain why a patient was flagged.

5.2 Limitations of the Model

Figure 5.2.1

Evidence for Class Imbalance, Data Leakage, and Overfitting

```
# Evidence for limitations (imbalance, leakage, overfitting)
import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score

FEATURES = feature_cols

if not isinstance(X_train, pd.DataFrame):
    X_train_df = pd.DataFrame(X_train, columns=FEATURES)
else:
    X_train_df = X_train

if not isinstance(X_test, pd.DataFrame):
    X_test_df = pd.DataFrame(X_test, columns=FEATURES)
else:
    X_test_df = X_test

print('1) CLASS IMBALANCE')
print(f'    Original dataset readmission rate: {y.mean():.2%}')

print('2) DATA LEAKAGE CHECKS')

overlap = X_train_df.merge(X_test_df.drop_duplicates(), how='inner', on=FEATURES)
print(f'    Identical patients in both train and test: {len(overlap)}')

single_auc = {}
for c in FEATURES:
    a = roc_auc_score(y_train, X_train_df[c])
    single_auc[c] = max(a, 1 - a)
suspicious = {c: round(a, 3) for c, a in single_auc.items() if a > 0.95}
print(f'    Features with single-feature AUC > 0.95: {suspicious or "none"}')
print('    Scaling and feature selection were fit on training data only.')
print('    Note: missing-value imputation and outlier capping used the full dataset '
      'before the train/test split - a mild form of leakage, small in magnitude.\n')

print('3) OVERFITTING CHECK (train vs CV vs test AUC)')

comparison = pd.DataFrame(results).T
comparison['overfit_gap'] = np.nan

for name in comparison.index:
    if 'train_auc' in comparison.columns:
        pass

from sklearn.metrics import roc_auc_score as _auc
for name, model in models.items():
    try:
        train_prob = model.predict_proba(X_train_scaled)[0, 1]
        comparison.loc[name, 'train_auc'] = _auc(y_train, train_prob)
    except Exception:
        comparison.loc[name, 'train_auc'] = np.nan

comparison['overfit_gap'] = comparison.get('train_auc', np.nan) - comparison['auc_roc']
comparison['flag'] = np.where(
    comparison['overfit_gap'] > 0.05, 'possible overfitting', 'ok'
)

show_cols = [c for c in
              ['train_auc', 'overfit_gap', 'auc_roc', 'accuracy', 'f1_score',
               'precision', 'recall', 'flag']]
if c in comparison.columns:
    display(comparison[show_cols].round(4))
```

```
... 1) CLASS IMBALANCE
    Original dataset readmission rate: 49.97%
2) DATA LEAKAGE CHECKS
    Identical patients in both train and test: 92
    Features with single-feature AUC > 0.95: none
    Scaling and feature selection were fit on training data only.
    Note: missing-value imputation and outlier capping used the full dataset before the train/test split - a mild form of leakage, small in magnitude.
3) OVERFITTING CHECK (train vs CV vs test AUC)
```

	train_auc	overfit_gap	auc_roc	accuracy	f1_score	precision	recall	flag
Logistic Regression	0.8753	-0.0003	0.8755	0.7915	0.7903	0.7944	0.7862	ok
Decision Tree	0.9117	0.1138	0.7979	0.7327	0.7319	0.7336	0.7302	possible overfitting
Random Forest	1.0000	0.1400	0.8600	0.7759	0.7737	0.7808	0.7667	possible overfitting
XGBoost	0.9561	0.1022	0.8538	0.7664	0.7662	0.7663	0.7661	possible overfitting

This code implementation exhibits three checks: the class balance of the target, two leakage tests covering train-test overlap and single-feature AUC, and a train versus test AUC comparison flagging gaps above 0.05.

The dataset is balanced at 49.97 percent, so resampling was unnecessary. No feature exceeded a single-feature AUC of 0.95, ruling out an outcome-encoding variable. However, the code output has

provided a notable form of mild leakage because of imputation and outlier capping done prior to train/test split. Addressing overlapping, Logistic Regression shows a train-test gap of -0.0003 and was flagged “ok”, leading to no overfitting. On the other hand, Decision Tree, Random Forest, and XGBoost all exceed the 0.5 threshold, with Random Forest reaching a perfect training AUC of 1.0000.

5.3 Bias detection & fairness audits

5.3.1 and 5.3.2 Fairness Assessment Across Sensitive Groups

Figure 5.3.1.1

Base Rate, Flagging Rate, and Error Rates by Age, BMI, and Diabetes Status

```
# Outcomes and fairness metrics across groups (test set)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

if not isinstance(X_test, pd.DataFrame):
    X_test_df = pd.DataFrame(X_test, columns=feature_cols)
else:
    X_test_df = X_test

try:
    X_test_scaled
except NameError:
    X_test_scaled = scaler.transform(X_test_df)
    print("Rebuilt X_test_scaled from scaler.transform(X_test_df).")

def add_groups(frame):
    out = pd.DataFrame(index=frame.index)
    out['age_group'] = pd.cut(
        frame['age'], bins=[0, 40, 60, 75, 200],
        labels=['<40', '40-59', '60-74', '75+'], right=False,
    )
    out['bmi_group'] = pd.cut(
        frame['bmi'], bins=[0, 18.5, 25, 30, 40, 200],
        labels=['Underweight', 'Normal', 'Overweight', 'Obese', 'Severe obesity'],
        right=False,
    )
    out['diabetes_group'] = frame['has_diabetes'].map({0: 'No diabetes', 1: 'Diabetes'})
    return out

audit_df = add_groups(X_test_df)
audit_df['y_true'] = y_test.values
audit_df['y_prob'] = best_model.predict_proba(X_test_scaled[:, 1])
audit_df['y_pred'] = (audit_df['y_prob'] >= 0.5).astype(int)

def group_rates(d, group_col, pred_col='y_pred'):
    rows = []
    for g, sub in d.groupby(group_col, observed=True):
        tn, fp, fn, tp = confusion_matrix(
            sub['y_true'], sub[pred_col], labels=[0, 1]
        ).ravel()
        rows.append({
            'group': str(g),
            'n': len(sub),
            'base_rate': sub['y_true'].mean(),
            'flag_rate': sub[pred_col].mean(),
            'TPR': tp / (tp + fn) if tp + fn else np.nan,
            'FPR': fp / (fp + tn) if fp + tn else np.nan,
            'PPV': tp / (tp + fp) if tp + fp else np.nan,
            'accuracy': (tp + tn) / len(sub),
        })
    return pd.DataFrame(rows).set_index('group')

def fairness_summary(rates):
    tpr_gap = rates['TPR'].max() - rates['TPR'].min()
    fpr_gap = rates['FPR'].max() - rates['FPR'].min()
    return {
        'demographic_parity_diff': rates['flag_rate'].max() - rates['flag_rate'].min(),
        'disparate_impact_ratio': rates['flag_rate'].min() / rates['flag_rate'].max(),
        'equal_opportunity_diff': tpr_gap,
        'fpr_diff': fpr_gap,
        'equalised_odds_diff': max(tpr_gap, fpr_gap),
    }

summary_rows = {}
for col in ['age_group', 'bmi_group', 'diabetes_group']:
    rates = group_rates(audit_df, col)
    print(f'\n{col}')
    display(rates.round(3))
    if (rates['n'] < 30).any():
        print(' Warning: some groups have fewer than 30 patients; their rates are unstable.')
    summary_rows[col] = fairness_summary(rates)

fairness_table = pd.DataFrame(summary_rows).T
fairness_table['four_fifths_rule'] = np.where(
    fairness_table['disparate_impact_ratio'] >= 0.80, 'pass', 'fail'
)
fairness_table['equalised_odds_target'] = np.where(
    fairness_table['equalised_odds_diff'] <= 0.10, 'met', 'not met'
)
print('\nFAIRNESS SUMMARY')
display(fairness_table.round(3))

age_rates = group_rates(audit_df, 'age_group')
age_rates[['base_rate', 'flag_rate', 'TPR', 'FPR']].plot.bar(figsize=(10, 4.5), rot=0)
plt.title(f'(best_model_name): outcomes by age group (test set)')
plt.ylim(0, 1)
plt.ylabel('Rate')
plt.tight_layout()
plt.show()
```

The code is utilized to determine bias detection and execute fairness metrics across the sensitive variables, structured in four stages. It first rebuilds the scaled test set and attaches the true outcome, the predicted probability, and the predicted label at a 0.5 threshold. It then derives three sensitive groupings from the available features, namely age in four bands, BMI in five clinical bands, and diabetes status as a binary split. For each group it computes the base rate, flagging rate, true and false positive rates, positive predictive value, and accuracy, then summarises these into demographic parity difference, disparate impact ratio, equal opportunity difference, and equalised odds difference. Each metric is scored pass or fail against its threshold, with disparate impact judged on the 80 percent rule and the odds differences on a 0.10 limit, and groups with fewer than 30 patients flagged as unstable.

5.3.3 Mitigation and Comparative Testing

Figure 5.3.3.1

Code Implementation of Reweighting and Age-Group Threshold Mitigation

```
# Mitigation, tested and compared (focus: age group)
from sklearn.base import clone
from sklearn.model_selection import cross_val_predict, StratifiedKFold
from sklearn.metrics import (
    roc_auc_score, accuracy_score, recall_score, precision_score
)

FEATURES = feature_cols
Xtr = X_train_scaled
Xte = X_test_scaled
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

if not isinstance(X_train, pd.DataFrame):
    X_train_df = pd.DataFrame(X_train, columns=FEATURES)
else:
    X_train_df = X_train

age_train = add_groups(X_train_df)['age_group']
age_test = audit_df['age_group']

p_group = age_train.value_counts(normalize=True)
p_y = y_train.value_counts(normalize=True)
p_joint = pd.crosstab(age_train, y_train, normalize=True)

weights = np.array([
    p_group[g] * p_y[yv] / p_joint.loc[g, yv]
    for g, yv in zip(age_train, y_train)
])

model_reweighted = clone(best_model)

try:
    model_reweighted.fit(Xtr, y_train, sample_weight=weights)
except TypeError as e:
    print(f"fit(sample_weight=...) failed ({e}); trying fit(..., sample_weight=...) via set_params")
    model_reweighted.fit(Xtr, y_train, sample_weight=weights)

prob_rw = model_reweighted.predict_proba(Xte[:, 1])

oof_prob = cross_val_predict(
    clone(best_model), Xtr, y_train, cv=cv, method='predict_proba'
)[:, 1]

target_tpr = recall_score(y_train, (oof_prob >= 0.5).astype(int))

group_thresholds = {}
for g in age_train.cat.categories:
    m = (age_train == g).values
    if m.sum() == 0:
        continue
    candidates = np.arange(0.05, 0.96, 0.01)
    tprs = np.array([
        recall_score(y_train[m], (oof_prob[m] >= t).astype(int))
        for t in candidates
    ])
    group_thresholds[g] = float(candidates[np.argmax(np.abs(tprs - target_tpr))])
```

```

thr_test = (
    age_test.astype(str)
    .map({str(k): v for k, v in group_thresholds.items()})
    .values
)
pred_thr = (audit_df['y_prob'].values >= thr_test).astype(int)

print(f'Target TPR (overall, out-of-fold): {target_tpr:.3f}')
print(f'Age-group thresholds:', {k: round(v, 2) for k, v in group_thresholds.items()})

def evaluate(pred, prob, label):
    d = audit_df[['age_group', 'y_true']].copy()
    d['pred'] = pred
    fs = fairness_summary(group_rates(d, 'age_group', 'pred'))
    return {
        'approach': label,
        'AUC': roc_auc_score(y_test, prob),
        'accuracy': accuracy_score(y_test, pred),
        'recall': recall_score(y_test, pred),
        'precision': precision_score(y_test, pred),
        **fs,
    }

mitigation = pd.DataFrame([
    evaluate(audit_df['y_pred'].values, audit_df['y_prob'].values, 'Baseline (threshold 0.5)'),
    evaluate((prob_rw >= 0.5).astype(int), prob_rw, 'A: Reweighting'),
    evaluate(pred_thr, audit_df['y_prob'].values, 'B: Age-group thresholds'),
]).set_index('approach')

mitigation.to_csv('fairness_mitigation.csv')
display(mitigation.round(3))

fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))

mitigation[['equal_opportunity_diff', 'fpr_diff', 'equalised_odds_diff']].plot.bar(
    ax=axes[0], rot=0
)
axes[0].axhline(0.10, ls='--', color='grey')
axes[0].set_title('Fairness gaps across age groups (lower is fairer)')

mitigation[['AUC', 'accuracy', 'recall']].plot.bar(ax=axes[1], rot=0)
axes[1].set_ylim(0.5, 1)
axes[1].set_title('Performance cost of each approach')

for ax in axes:
    ax.tick_params(axis='x', labelsize=8)

plt.tight_layout()
plt.show()

```

The code implements two mitigation approaches targeting age. Reweighting computes sample weights from the joint distribution of age group and outcome, then refits the model with those weights. Age-group thresholds leave the model unchanged and instead select, for each group, the cut-off whose true positive rate comes closest to the overall target obtained by cross-validation. Both are evaluated against the baseline on the same test set.

Table 5.3.3.2

Fairness Metrics of Each Mitigation Approach

Approach	Demographic parity diff	Disparate impact ratio	Equal opportunity diff	FPR diff	Equalised odds diff
Baseline (threshold 0.5)	0.273	0.572	0.144	0.096	0.144
A: Reweighting	0.124	0.781	0.020	0.063	0.063
B: Age-group thresholds	0.124	0.780	0.028	0.068	0.068

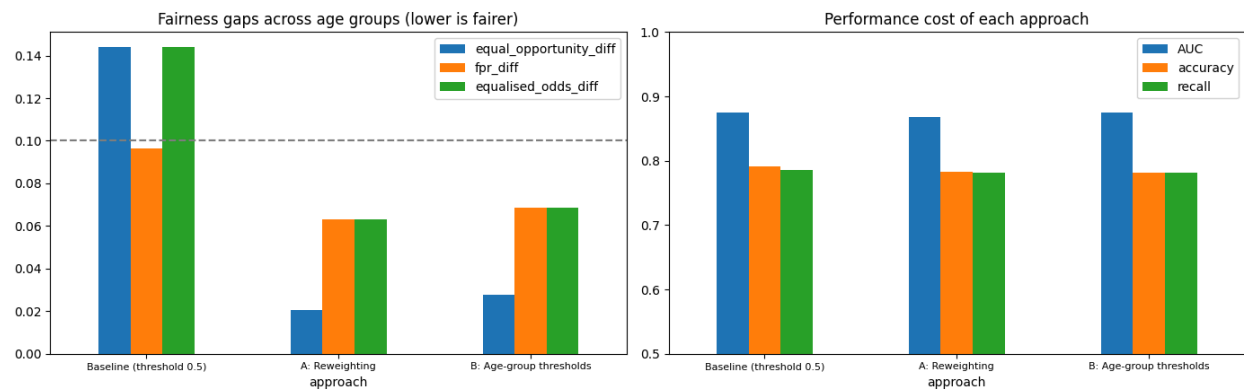
Approach	AUC	Accuracy	Recall	Precision
Baseline (threshold 0.5)	0.876	0.791	0.786	0.794
A: Reweighting	0.867	0.783	0.781	0.784

Approach	AUC	Accuracy	Recall	Precision
B: Age-group thresholds	0.876	0.782	0.781	0.782

Both mitigations reduce the fairness gaps substantially. Reweighting cuts the equalised odds difference from 0.144 to 0.063 and raises disparate impact from 0.572 to 0.781, costing 0.009 AUC. Age-group thresholds perform comparably while preserving AUC at 0.876. Reweighting is adopted, since it achieves the lowest equal opportunity difference at 0.020 and applies one threshold to all patients.

Figure 5.3.3.3

Graphs of Fairness Gaps and Performance Cost Across Mitigation Approaches



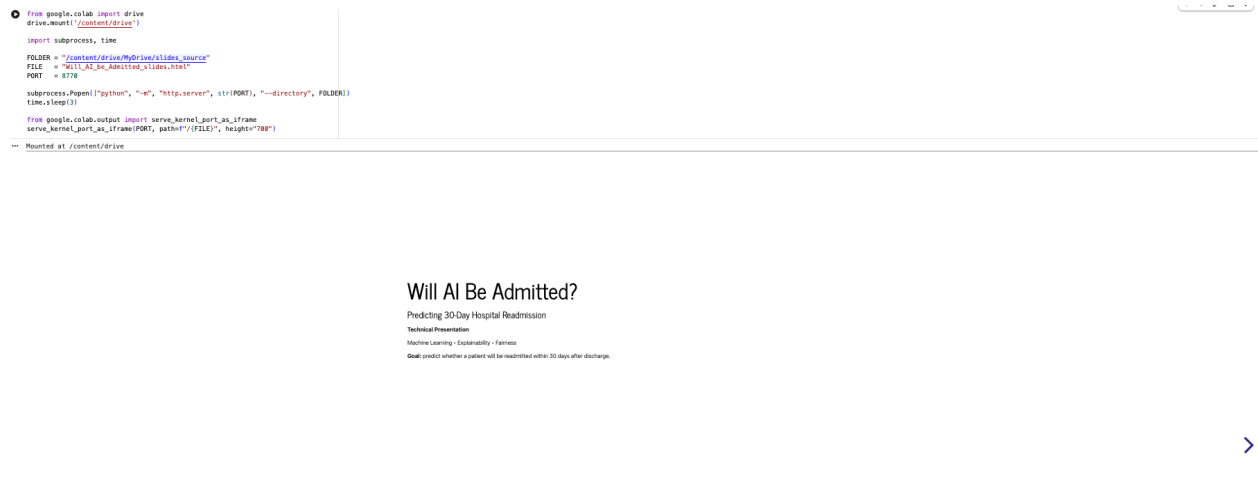
The left panel shows all three gaps falling below the 0.10 threshold after mitigation, from a baseline of 0.144 for equal opportunity and equalised odds. The right panel shows performance holding nearly constant across all three approaches, confirming the fairness gain came at negligible cost.

VI. Final Presentation & Communication

6.1 Two deliverables for mixed audiences:

Figure 6.1.1

Process of Technical Presentation Execution



The technical presentation consists of the summarised content of the project and was produced using Jupyter Notebook, exported to HTML slides through the RISE extension. It is addressed to a peer audience and follows the sequence of the notebook itself, covering the problem framing, dataset, preprocessing, key exploratory findings, modelling setup, model comparison, explainability, and the fairness audit, using the figures generated in the notebook rather than reproduced separately.

➔ Business Facing Presentation: <https://canva.link/mrffvbwctv4l4l6>.

The business-facing presentation consists of the same findings translated for an executive audience, produced in Canva. It covers eight sections: the problem and its significance, the project objectives, the data and methods used, the results of the model comparison, explainability in terms of what drives a prediction, the fairness audit, the bias mitigation tested, and the contribution and access arrangements. The emphasis is on the decision being asked for and the risks involved rather than on how the model was built.

VII. GitHub Profile & Upload

7.1 Create a public GitHub repo structured like an Open Source project:

Figure 7.1.1:

Repository Landing Page

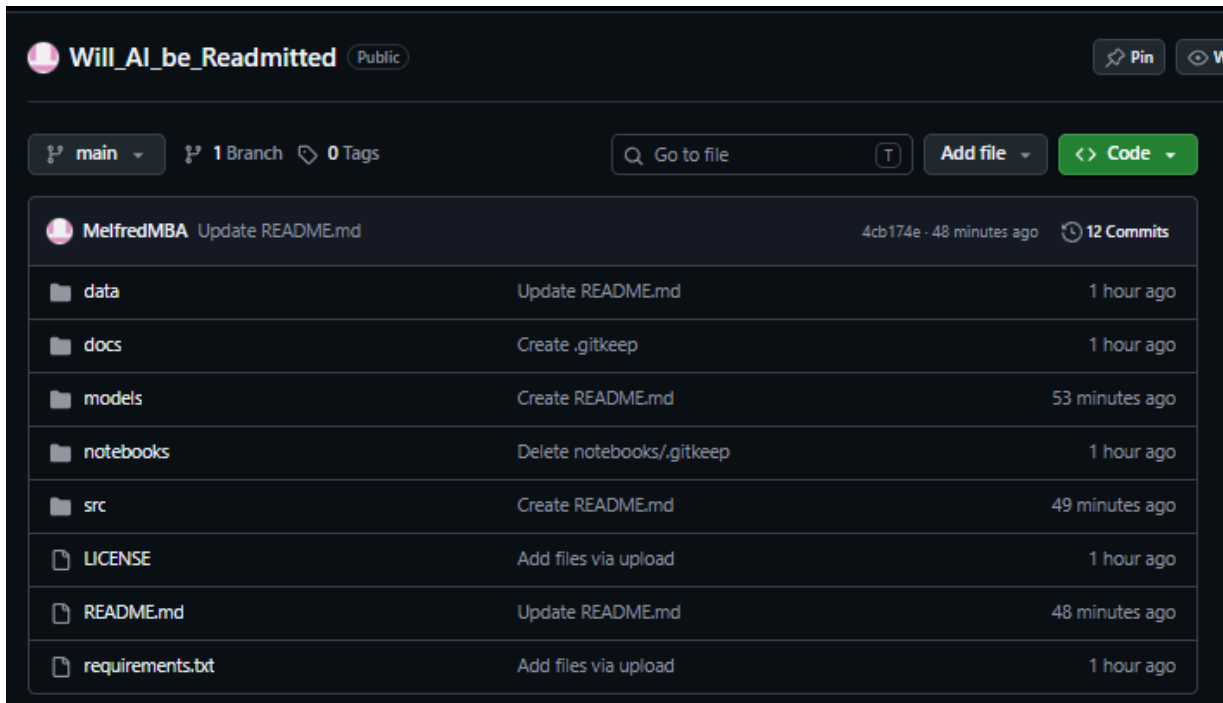


Figure 7.1.2:

Root Repository README preview

Programmer: Melfred P. Sumaya, PTRP, MBA **Project:** Will AI be Readmitted?: An AI-assisted hospital readmission assessment.

This project predicts whether a patient is likely to be readmitted to the hospital within 30 days of discharge using previous information from the user. It focuses on using age, BMI, previous admissions, diabetes status, and comorbidities.

Features

- Interactive web form for entering patient details
- Real-time risk output (LOW / MODERATE / HIGH) with probability according to threshold.
- BMI, comorbidity score, and diabetes status are the main factors
- Shows the top risk drivers behind the prediction
- Fairness audit across age, BMI, and diabetes groups
- Bias mitigation tested with reweighting and group thresholds

Program Stack

Backend

- Flask (Python)

Frontend

- HTML, CSS (W3.CSS), JavaScript

Machine Learning

- scikit-learn (Logistic Regression, Decision Tree, Random Forest)
- XGBoost (used for comparison only)
- Final model: Logistic Regression
- Explainability: SHAP, Partial Dependence Plots (PDP + ICE)

Fairness

- Custom metrics: Demographic Parity, Disparate Impact (EEOC 80% rule), Equal Opportunity, Equalized Odds to address different scenarios of people.

Deployment

- Google Colab + ngrok

Environment

- Python 3.10+, Jupyter Notebook or Google Colab

Contributors 1

MelfredMBA

Languages

Jupyter Notebook 97.4% Python 1.4% HTML 1.2%

Suggested workflows

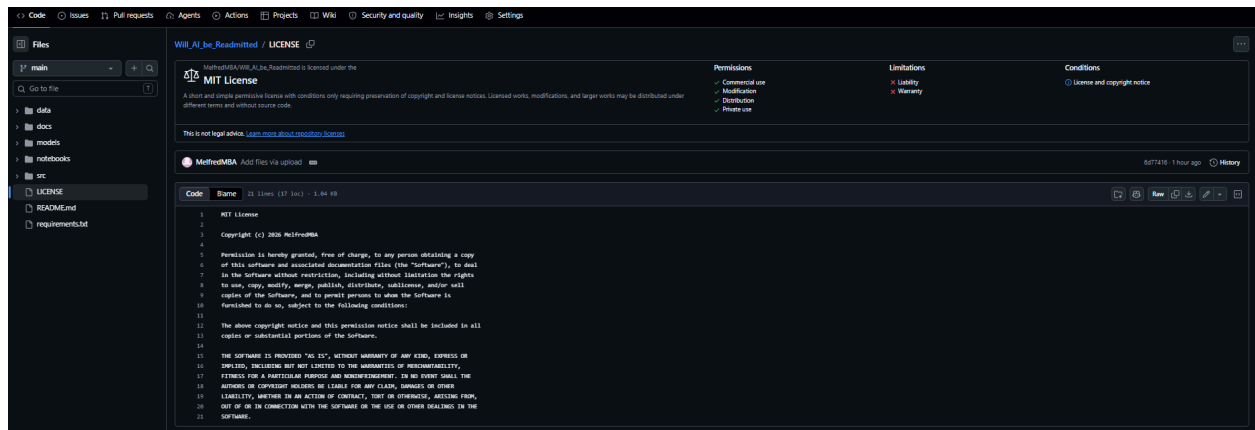
Based on your tech stack

- SLSA Generic generator** Generate SLSA3 provenance for your existing release workflows By Open Source Security Foundation (OpenSSF) [Configure](#)
- Python application** Create and test a Python application. By GitHub Actions [Configure](#)
- Jekyll using Docker image** Package a Jekyll site using the jekyll/builder Docker image. By GitHub Actions [Configure](#)

[More workflows](#) ✕

Figure 7.1.3:

License File for Public repositories



The repository can be publicly accessed via https://github.com/MelfredMBA/Will_AI_be_Readmitted. This follows the standard open-source requirements: descriptive project name, intuitive and public sequence of folders, an MIT LICENSE, and README.md providing an overview of how the whole repository works, covering installation and usage guide. The requirements.txt included provides which dependencies are needed. Changes can be tracked via Git with descriptive commit messages.

7.2 Include: src/ for scripts, notebooks/, data/, models/ directories and 7.3 Deliverables: GitHub repo link + final report + reproducible code.

Figure 7.2.1:

Root view with all the four folders required

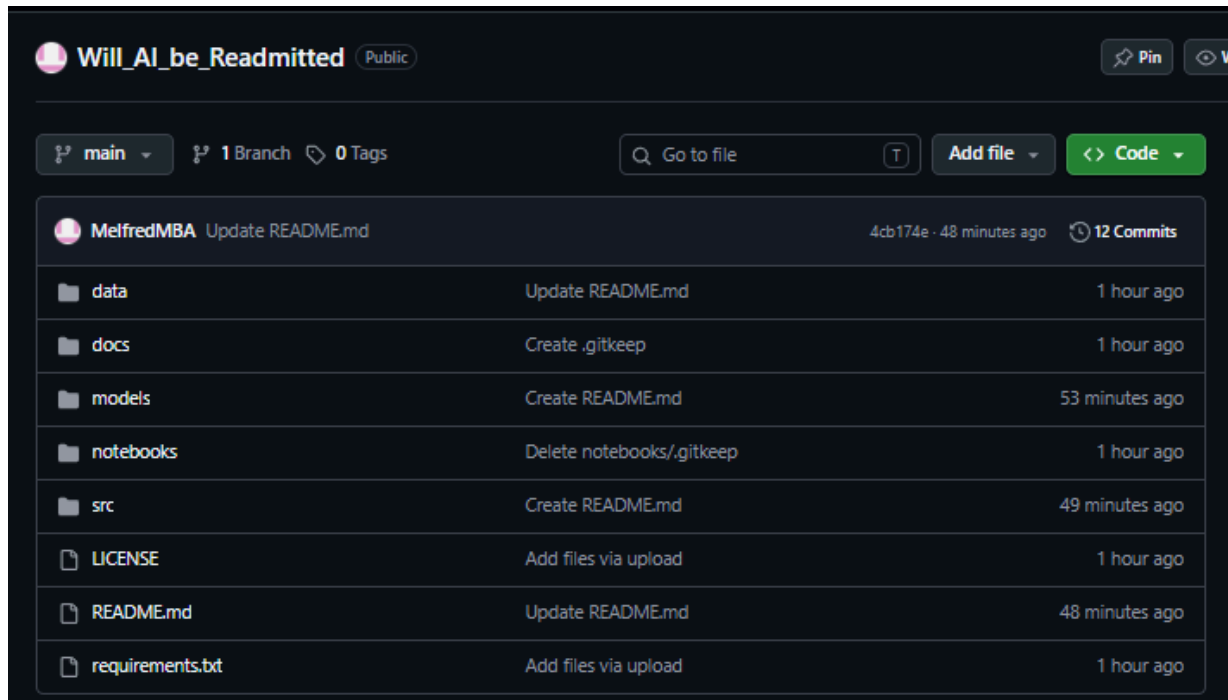


Figure 7.2.2:

src/ view with README preview for guide

The screenshot shows a file explorer interface with a dark theme. On the left, a sidebar titled 'Files' shows a tree view of the project structure. The 'src' directory is selected, showing its contents: README.md, app.py, fairness_mitigation.py, model.py, preprocess.py, LICENSE, README.md, and requirements.txt. The main area on the right displays the 'src' directory path and a commit history table. Below the table, the README.md file is previewed, showing its content.

Files

main

Go to file

- data
- docs
- models
- notebooks
- src**
 - templates
 - README.md
 - app.py
 - fairness_mitigation.py
 - model.py
 - preprocess.py
 - LICENSE
 - README.md
 - requirements.txt

Will_AI_be_Readmitted / src

MelfredMBA Create README.md aa61dc4 · 29 minutes ago History

Name	Last commit message	Last commit date
..		
templates	Add files via upload	1 hour ago
README.md	Create README.md	29 minutes ago
app.py	Add files via upload	1 hour ago
fairness_mitigation.py	Add files via upload	1 hour ago
model.py	Add files via upload	1 hour ago
preprocess.py	Add files via upload	1 hour ago

README.md

This folder contains the Python source scripts and Flask web apps.

Files:

1. app.py - Flask server, loads the model from the models folder. This also exposes the API routes.
2. fairness_mitigation.py - This is where reweighting and group thresholds occurs.
3. model.py - Model training utilities.
4. preprocess.py - Preprocessing utilities.
5. templates/index.html - the HTML file being launched through Flask.

Figure 7.2.3:

src/templates/ view

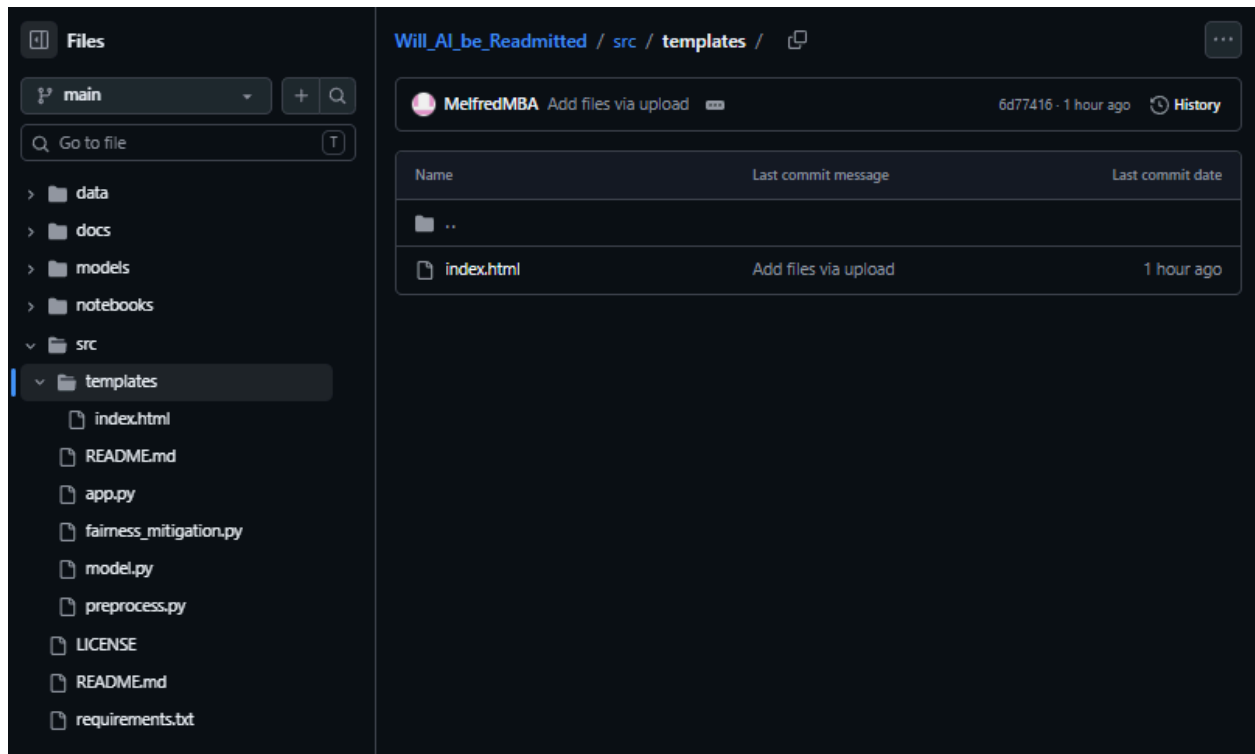


Figure 7.2.4:

notebooks/ view

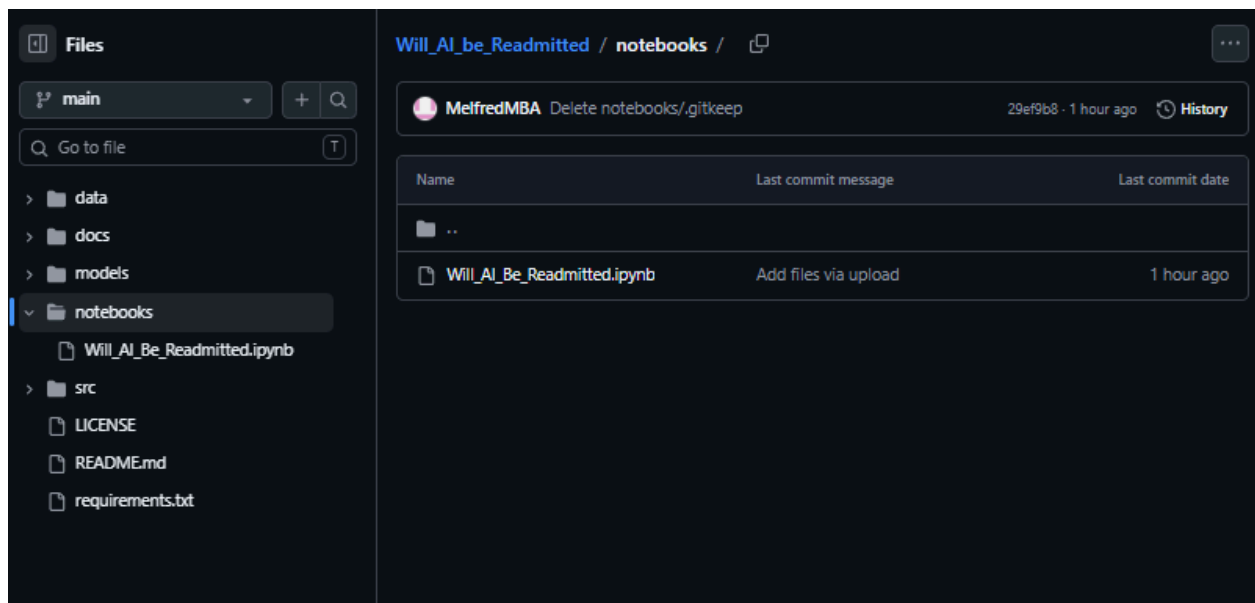


Figure 7.2.5:

data/ view with README preview for guide

The screenshot shows a file explorer interface with a dark theme. On the left, a sidebar titled 'Files' shows a tree view with a 'main' branch and a 'data' folder. The 'data' folder is expanded, showing files: README.md, data_dictionary.csv, and hospital_readmission.csv. Below these are folders: docs, models, notebooks, and src, followed by files: LICENSE, README.md, and requirements.txt. The main area on the right shows the path 'Will_AI_be_Readmitted / data /' and a commit history for 'MelfredMBA' with the message 'Update README.md' from 37 minutes ago. Below this is a table of files in the 'data' directory:

Name	Last commit message	Last commit date
..		
README.md	Update README.md	37 minutes ago
data_dictionary.csv	Add files via upload	1 hour ago
hospital_readmission.csv	Add files via upload	1 hour ago

Below the table is a preview of the README.md file, titled 'README.md' with an edit icon. The content of the README.md file is as follows:

Files in this folder:

- hospital_readmission.csv - Raw dataset obtained through Kaggle.
- data_dictionary.csv - variable names, types, units, allowed values.

Source for dataset:

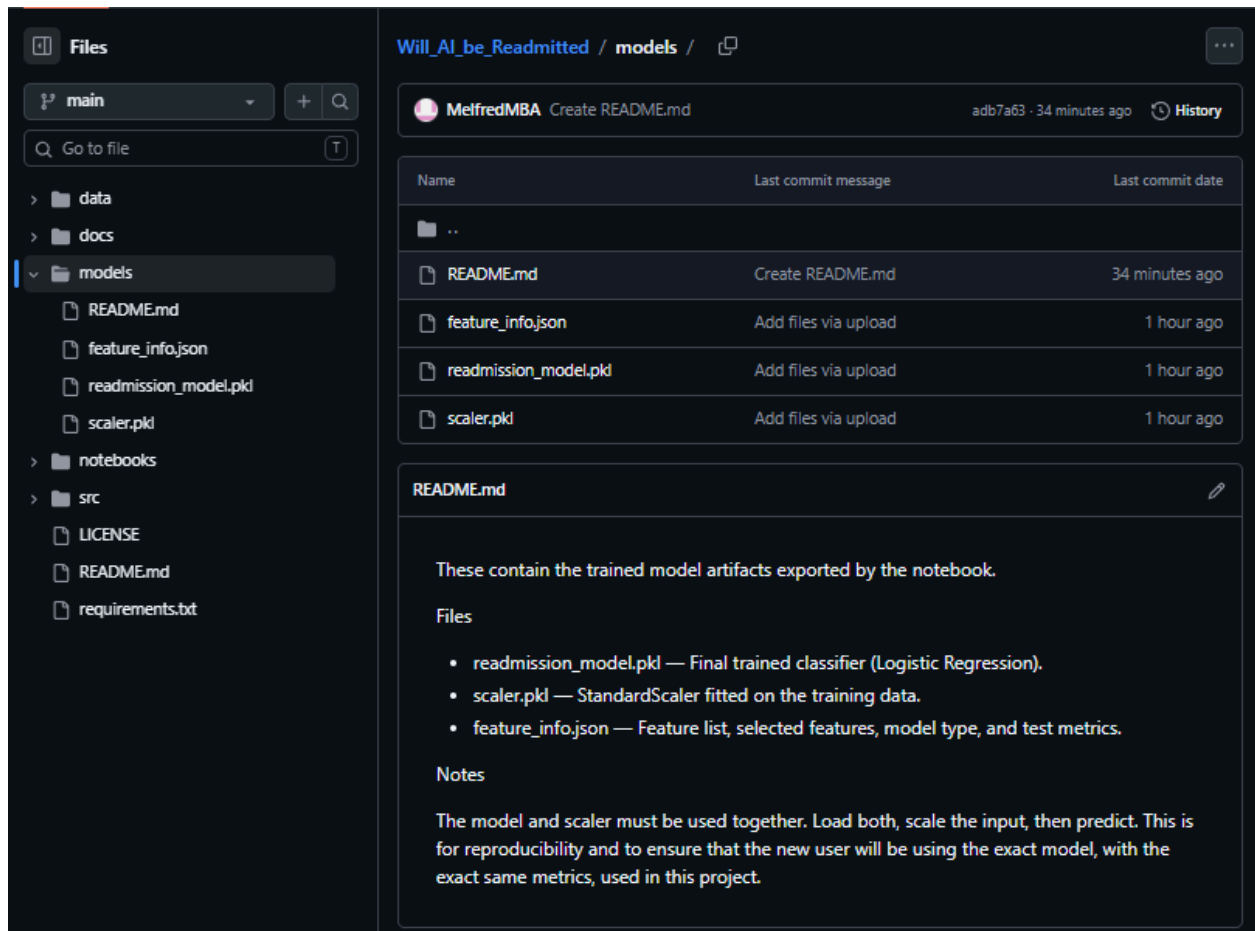
- <https://www.kaggle.com/datasets/abhaypanchal/hospital-readmission-prediction-dataset?resource=download>

How to reproduce:

- First obtain the notebook from Kaggle, then mount it in Google Drive.
- Strictly follow the directory path from the code when reproducing.
- If running locally, modify where the notebook points to (see the notebook note).

Figure 7.2.6:

models/ view with README preview for guide



The repository is organized into four functional directories:

- ❖ src/ - This directory provides the sources scripts including Flask application, fairness mitigation module, training and preprocessing utilities, and the frontend used in the study.
- ❖ notebooks/ - This provides the full end-to-end notebook used in the study.
- ❖ data/ - The raw dataset and the data dictionary is found in this directory.
- ❖ models/ - The saved training model and the supporting files.
- ❖ docs/ - This shows the uploaded documentation of the whole study.

This can all be accessed via https://github.com/MelfredMBA/Will_AI_be_Readmitted. The code can be immediately reproduced via using the notebook in the notebooks/ directory through Google Colab or Jupyter. Ensure strict compliance on the directory format for the dataset mounting, the notes in the code

have provided an appropriate approach according to whether the user would run it through Google Colab or Jupyter.

VIII. (Optional) Complete this step if you have a good understanding of model deployment and MLOps practices.

8.1 Local Deployment (required): deploy the best model via Flask, FastAPI, or Dash.

Figure 8.1.1

Flask Application Setup Code Implementation

```
# Flask dependencies.

!pip install flask flask-cors pyngrok -q

from flask import Flask, request, jsonify, render_template_string
from flask_cors import CORS
from pyngrok import ngrok
import threading
import time

# Load flask, open the website and create tunnel using CORS. Load trained models as well.
app = Flask(__name__)
CORS(app)

# Load saved artifacts
with open('readmission_model.pkl', 'rb') as f:
    model = pickle.load(f)
with open('scaler.pkl', 'rb') as f:
    scaler = pickle.load(f)
with open('feature_info.json', 'r') as f:
    feature_info = json.load(f)

# Store for session data
sessions = {}
```

The final model is launched through the Flask application. This can be run locally through the provided files with python app.py found in the GitHub repository. This study is demonstrated in Google Colab where it exposed a publicly available URL via ngrok tunnel for evaluation. The same code supports both local and temporary public access.

8.3 MLOps practices:

A. Reproducible environments (requirements.txt, Docker).

The requirements.txt file has provided all the Python dependencies with the project. The notebook and source scripts use a fixed random seed (SEED = 42) to ensure reproducibility and consistent outcomes of training results. A Dockerfile was not used in the study.

B. Config-driven runs & experiment tracking (MLflow, W&B, etc.).

While this was not implemented, the study has provided model comparison results exported via fairnes_mitigation.csv and feature_info.json.

C. CI checks (lint, unit tests), basic monitoring plan.

CI check and monitoring was not implemented as the study focuses on manual notebook execution for validation. No continuous-integration pipeline or automated test suite was used. On

the other hand, the notebook has basic monitoring provided in the Flask app through the /health endpoint which reports whether the model has been loaded or not.

Additionally, the notebook, found in the last cell, has provided a log-viewing cell to keep the Flask server running while printing incoming requests to the console. This method allows the administrator or project handler to observe GET and POST interactions in real time and detect malfunctions during a session. This approach provides a lightweight form of runtime monitoring, though not a substitute if the project is deployed completely in a 24/7 setup.

D. Version & rollback plan.

The source code and notebook versions are tracked via Git and all are provided in the GitHub repository. The trained model artifacts are versioned by re-running the notebook and re-committing. A formal rollback procedure was not documented, if rollback is done, the user would have to check an earlier commit and re-run the export cell.

8.4 Provide a demo (GIF/screencast).

The video demonstration provides an execution providing the three categories of readmission rates in the study. The video demonstration is accessible via this link: <https://canva.link/dmhw1594q10oeck>.